

Documentación Proyecto 4

En este documento se detallará la ideación, diseño e implementación de los cuatro códigos .asm respectivos del proyecto 4 y los dos códigos que son extensión del curso: Fill.asm, Mult.asm, SumN.as y CopyBlock.asm. Para la realización de todos los entregables se empleó como base códigos o explicaciones realizadas en clase.

Para realizar **Fill.asm** lo primero fue estudiar las diapositivas de clase donde se explicaba el papel de la variable KBD y como se emplea en el código para detectar si hay una tecla presionada o no, este fue el primer bloque de código creado, dentro de la sección '(STOP)', que redirigía a un '(LOOP)' cuando se detectaba la tecla presionada, es decir, cuando la variable KBD fuese diferente de 0. Posteriormente la lógica a implementar fue la de dicho '(LOOP)', dentro de este se debe pintar cada píxel de la pantalla y se utilizó la siguiente lógica:

Primero, para realizar el pseudocódigo, la premisa fue: LOOP por filas, LOOP por columnas, resultando en el siguiente bosquejo:

```
If (var != 0){  
    for(int i=16384; i<=24575; i++){  
        for(int j=0; j<32; j++){  
            RAM[i][j] = -1;}  
        }  
    }  
}
```

Esta idea se tradujo en el código .asm con un contador, el cual debía llevar el control de ir avanzando posición por posición hasta cubrir las 8192 posiciones; debido a que las posiciones de memoria para 'screen' comienzan en la posición 16384 hasta la 24575, es decir, 8191 posiciones, y se le suma 1 ya que comenzamos sumando desde el 0 (para pintar también la posición inicial). Y ya en cada posición nueva calculada se debe igualar su contenido a -1 (M=-1) ya que este valor es el encargado de poner los pixeles en color negro.

Hasta el momento hemos logrado que al identificar una tecla presionada la pantalla se ponga en negro, pero es también necesario que la pantalla vuelva al color blanco una vez que la tecla deje de ser presionada. Para esto se emplea un nuevo '(LOOP)' llamado '(BLANCO)' que sigue la misma lógica del contador, pero ahora se coloca en el contenido de cada posición el valor 0 (M=0), encargado del color blanco.

Realizar estos bloques, que equivalen a los ciclos, fue relativamente fácil y nos basamos mucho en los códigos de ejemplo que realizamos en clase. Sin embargo, la dificultad fue lograr sincronizar los ciclos, es decir, que la respuesta al momento de dejar presionada la tecla no fuera que la pantalla cambiara de blanco a negro titilando (como si se estuviera presionando continuamente y no como si estuviera sosteniendo), o que se devolviera a blanco inmediatamente la tecla dejara de ser presionada.

Para lograrlo, la variable contador se va a reiniciar en un ciclo adicional llamado '(RESET)' donde lo único que se define es colocar en 0 la variable contador y luego un salto a '(STOP)' para seguir inspeccionando la salida del teclado. El salto hacia '(RESET)' se realiza siempre al final de '(LOOP)' y de '(BLANCO)', una vez que la variable contador es igual al total de pixeles a pintar. Esto asegura que no titile la pantalla y que el cambio de negro a blanco sea inmediato una vez que la tecla deje de ser oprimida, debido a que en ambos bucles siempre se asegura que se pinta la totalidad de la pantalla antes de realizar nuevamente alguna evaluación a cualquier variable.

Cuando en '(RESET)' se realiza el salto hacia '(STOP)' lo siguiente a evaluar es si hay alguna tecla presionada, en caso de estarlo, vuelve desde 0 a pintar toda la pantalla de negro, es por esto por lo que, en caso de que la tecla nunca se haya dejado de presionar, no se verá una diferencia porque ya toda la pantalla estaba negra y lo único que se hace es volver a pintar cada píxel, es decir, nunca dejan de estar de color negro. Por otro lado, si no hay teclas presionadas, pasará inmediatamente a pintar en blanco la pantalla ya que el bloque de código '(BLANCO)' se en cuenta inmediatamente después del salto a '(LOOP)', por ende, de no realizarse dicho salto, el código continúa ejecutándose en cascada.

Así logramos implementar el código Fill.asm de la manera en que se definía en los comentarios de instrucción del archivo, logrando que el código fuese un *'infinite loop'* que escuchara la entrada del teclado permanentemente y que al no haber ninguna tecla presionada se logre que *'the screen should be cleared'*.

La implementación de **Mult.asm** fue una reutilización parcial de un código creado en un ejercicio de clase y la lógica adicional era un poco más sencilla: con una variable 'sum' se va acumulando el resultado parcial de la suma en cada iteración, y con una variable de control 'i', vamos revisando si ya se llegó al final de iteraciones. La inicialización de estas dos variables en 0 se realiza una única vez en la parte inicial del código, justo antes del ciclo principal '(LOOP)'.

En '(LOOP)' se realiza toda la lógica de la suma repetitiva: primero se evalúa si el valor de R1 menos el valor de i es igual a 0, lo que significaría que ya se sumó R[1] veces el valor de R[0], en cuyo caso la variable sum almacena el resultado final que se debe guardar en R2,

por ende, se redirige a un '(LOOP)' llamada '(RESULT)'. Si no se cumple esta condición, y dicha resta es un número mayor a 0, se continua con la suma, extrayendo el contenido de R0 ($D=M$) y sumádoselo al valor actual de sum, posteriormente, se suma 1 a la variable i y se realiza un salto a '(LOOP)'. Hacer esta resta para verificar si ya se igual al valor de R1 asegura que la multiplicación por 0 (sea que el 0 este en R1 o en R0) sí de como resultado 0, ya que en caso de primero realizar la adición y luego verificar con la resta, no se cumpliría esta condición.

En el ciclo '(RESULT)' lo único que se hace es extraer el valor de la variable suma y se almacena en R2, para finalmente redirigir al ciclo '(END)', donde ya el programa se quedara iterando infinitamente, pero ya finalizado.

De una manera similar, para implementar **SumN.asm** se siguió la misma lógica de una variable 'sum' encargada de la suma parcial en cada iteración y también una variable 'i' de control, ambas inicializadas en 0. Una adición de este código es que antes del '(LOOP)' principal, además de inicializar las variables, también se realiza una resta entre el valor de R0 (cantidad de N números naturales a sumar) y la variable 'i' (en este momento con valor 0) para poder identificar si $R0=0$, de modo que inmediatamente salte al bucle '(STOP)', que se encarga de copiar el valor de sum (en este caso 0 porque nunca entro al ciclo principal) en R1 y terminar el programa en el ciclo '(END)'.

Ahora, si esta resta es diferente de 0, entra directamente en el ciclo '(LOOP)' al seguir el código en cascada y sin saltos. Dentro de este código la lógica es la misma que en Mult.asm, ya que se extrae el valor de 'i' y se le suma al valor de 'sum', para luego realizar la resta entre el valor de R0 y el de 'i' y en caso de que sea 0 saltará a '(STOP)', sino a 'i' se le sumará 1 y se saltará nuevamente a '(LOOP)'. Una vez el valor de 'i' iguale el valor de R0, entonces se saltará al bloque '(STOP)', explicando anteriormente y de allí al bloque '(END)' donde finaliza el programa.

Por último, para realizar el archivo de extensión del curso **CopyBlock.asm** se reutilizaron ciertas ideas de los códigos anteriores, como la de la variable 'i' como contador, en este caso para poder revisar en cada iteración si el valor de i es igual al valor de R2, que indica cuando ya se copiaron las n posiciones solicitadas (valor de R2). La lógica detrás de este código requiere un entendimiento claro de la diferencia entre el registro A y el M: el registro A de la CPU determina siempre la DIRECCIÓN DE MEMORIA a la que se está apuntando. Cuando A recibe un valor (por ejemplo, con $A=M$ tras consultar una variable, o después de un cálculo como $A=D+M$), la CPU pasa a apuntar a esa dirección específica de la RAM. Por otro lado, M representa siempre el contenido de la celda de memoria actualmente seleccionada por A ($RAM[A]$), de modo que se puedan leer y escribir datos en dicha posición.

Además de estos registros, vamos a enumerar las variables auxiliares que empleamos:

- **Posi:** por seguridad se almacenó el contenido de R0 en esta variable, que representa la posición de origen donde debe iniciar a BUSCAR el bloque de código a copiar. Es más seguro almacenarlo en una variable auxiliar en caso de que el contenido de R0 se sobrescriba por algún motivo.
- **Posf:** esta variable va a almacenar el contenido de R1, que representa la posición donde INICIARÁ a copiar el bloque de código. Al igual que con 'Posi', se emplea esta variable auxiliar por seguridad, además de que el nombre ayuda a recordar su función.
- **n:** esta variable contiene el valor almacenado en R2, que representa la CANTIDAD de posiciones a copiar, es decir, el tamaño del bloque.
- **Valor:** esta variable se encarga de almacenar el contenido de la posición origen que estamos copiando en cierta iteración, para luego almacenarlo en la posición final.
- **Dir_destino:** esta variable guarda EN SU CONTENIDO la dirección de memoria en la que voy a almacenar el contenido que guarde en 'valor', es decir, esta variable guarda la dirección de la posición final.

La interacción entre estas variables ocurre en el único ciclo (además del '(END)') que tiene este programa. Inicialmente se da el almacenamiento de los valores de R0, R1 y R2 en las variables posi, posf y n, además de la inicialización de 'i', la variable contadora, en 0. Posteriormente se llega al bucle '(LOOP)', donde lo primero será siempre revisar que la resta del contenido de n entre el contenido de i sea diferente de 0, para continuar en '(LOOP)'; de ser 0, entonces se redirige a '(END)' y se queda corriendo infinitamente allí de modo que se sabe que acabo el código, esto como buena práctica.

Posteriormente, se almacena el valor (el contenido) de la variable 'posi' en el registro D, para luego consultar el valor almacenado en i (@i) y sumar el valor de D con el contenido de i, y dicho valor será la dirección de memoria que contiene el dato a copiar, por lo que se igualará la suma a A ($A=D+M$). Al estar apuntando a dicha dirección, extraemos su contenido ($D=M$) y lo guardamos en la variable auxiliar 'valor' (@valor, $M=D$), de modo que ya tenemos en una variable el contenido que debemos copiar.

Para calcular la posición de destino, donde se debe copiar el valor que acabamos de almacenar, el proceso es idéntico al recuperar el valor almacenado en 'posf' (que es la dirección de memoria donde se debe copiar el dato), solo que en vez de apuntar a dicha posición y almacenar su contenido, únicamente guardamos la dirección de memoria final (contenido de posf + i) en la variable 'dir_destino'. De modo que en este

momento tenemos dos datos fundamentales: la dirección de memoria donde debemos almacenar el dato y el contenido que debemos guardar allí.

Ahora debemos realizar dicha operación: para ello consultamos el contenido de la variable valor (@valor, que tiene el contenido a guardar) y lo almacenamos en el registro D temporalmente, luego consultamos el contenido de la variable dir_destino (@dir_destino, que es la dirección de memoria) y apuntamos a ella ($A=M$), para posteriormente almacenar en ella el valor que teníamos en D ($M=D$). De modo que nos queda ya en la dirección de memoria final donde debíamos copiar el dato que era necesario trasladar.

Ya solo queda sumarle a la variable i una unidad, porque ya realizamos el proceso completo de copiar un dato. Después se realiza obligatoriamente un salto a '(LOOP)', para repetir el bucle, ya que el filtro que revisa si se han copiado todos los datos está al inicio del bucle. Después, lo único que queda es el último bucle '(END)' que es a donde se es redirigido cuando la variable n y la variable i tienen el mismo valor, lo que indica que ya se copiaron todos los datos y finalizó el programa, allí se queda en un bucle infinito y no sale de ahí.